XTrace Specification v1.0

AI-Native Debug Trace Format for SystemVerilog

Release: 2026-04-30

1. Introduction

XTrace is a debug trace format intended to replace or complement VCD when the consumer is not only a waveform viewer, but also a parser, debug database, formal or debug analysis tool, AI agent, or LLM-based root-cause assistant.

XTrace is designed to be compact, structured, semantic, streamable, easy to parse, easy to summarize, and better aligned with hardware transactions than raw signal toggles.

2. Design Goals

Compared to VCD, XTrace aims to provide:

1. dictionary-based naming to avoid repeating long hierarchy strings

2. delta-oriented signal dump so only changes are emitted

3. transaction-first modeling for requests, responses, instructions, bursts, exceptions, and cache activity

4. AI-friendly semantic records for assertions, events, causality, and context bundles

5. multi-level observability across waveform-level, transaction-level, and summary-level views

6. text and binary forms for easy bring-up and efficient deployment

3. Format Variants

3.1 XTrace-T

Human-readable text format. Recommended for bring-up, debugging the parser or emitter, diffing traces, and simple tool interoperability.

3.2 XTrace-B

Compact binary format. Recommended for long regressions, large SoCs, high-frequency traces, database ingest, and cloud processing.

This specification focuses mainly on XTrace-T while also defining the binary packet model.

4. Conceptual Model

XTrace consists of four layers:

4.1 Dictionary layer

Static metadata including modules, signals, enums, transaction schemas, tags, and optional source maps.

4.2 Signal delta layer

Time progression plus changed signal values.

4.3 Semantic event layer

Higher-level records such as instruction decode and retire, FSM transitions, assertion pass and fail, and reset or power transitions.

4.4 Transaction layer

Transaction lifecycle records for begin, update, end, abort, linkage, and summaries.

5. File Structure

An XTrace text file is composed of top-level directives followed by sections.

Example:

```
@xtrace 1.0
@format text
@timescale 1ns
@design cpu16
@profile architectural

@section dict
...
@section trace
...
@section transactions
...
```

```
@section end
```

6. Top-Level Directives

6.1 Version directive

```
@xtrace 1.0
```

Required. Declares the file format version.

6.2 Format directive

```
@format text
```

Allowed values: text, binary.

6.3 Timescale directive

```
@timescale 1ns
```

Defines the unit of timestamps and deltas.

6.4 Design directive

```
@design cpu16
```

Optional design name.

6.5 Profile directive

```
@profile architectural
```

Optional. Indicates intended debug scope. Recommended values: minimal, architectural, bus, microarch, full_debug.

7. Sections

The following sections are defined:

- @section dict
- @section trace
- @section transactions
- @section end

Optional future sections:

- @section summaries
- @section memory
- @section strings

8. Dictionary Records

The dictionary section defines all static entities used by later trace records.

8.1 Module record

Syntax:

```
M,<module_id>,<hier_path>
```

Example:

```
M,m0,/top
M,m1,/top/cpu
M,m2,/top/cpu/alu
```

Rules:

- module_id must be unique
- hier_path should use stable canonical hierarchy form
- module IDs are referenced by signal definitions

8.2 Signal record

Syntax:

```
S,<signal_id>,<module_id>,<name>,<type>[,tags=<tag1>|<tag2>|...][,enc=<mode>]
```

Example:

```
S,s0,m1,pc,u16,tags=pc|state,enc=delta
S,s1,m1,ir,u16,tags=instruction,enc=abs
S,s2,m1,state,enum:cpu_state,tags=fsm|state,enc=enum
```

```
S,s3,m1,mem_addr,u16,tags=req_addr|bus,enc=abs
```

```
S,s4,m1,mem_rdata,u16,tags=rsp_data|bus,enc=abs
```

```
S,s5,m1,mem_wr,bit,tags=control|bus
```

```
S,s6,m1,r1,u16,tags=register|architectural
```

Rules:

- signal_id must be unique
- module_id must refer to a declared module
- signal full path is reconstructed as module_path/name
- tags are optional but strongly recommended
- encoding mode is optional

8.3 Signal types

Recommended scalar and vector types:

- bit
- u8, u16, u32, u64
- s8, s16, s32, s64
- logic[N]
- enum:<enum_name>
- str
- bytes
- mem[addr_width,data_width]

8.4 Enum record

Syntax:

```
E,<enum_name>,<value0>=<name0>,<value1>=<name1>,...
```

Example:

```
E,cpu_state,0=RESET,1=FETCH,2=DECODE,3=EXEC,4=MEM,5=WB
```

8.5 Transaction schema record

Syntax:

```
TQ,<txn_type_id>,<txn_type_name>,fields=<f1>|<f2>|...[,tags=<t1>|<t2>|...]
```

Example:

```
TQ,tt0,inst,fields=pc|opcode|mnemonic|rd|result|status|latency,tags=instruction

TQ,tt1,mem_read,fields=addr|size|rdata|status|latency|parent,tags=bus|memory

TQ,tt2,mem_write,fields=addr|size|wdata|status|latency|parent,tags=bus|memory
```

Purpose:

- standardize transaction field sets
- reduce ambiguity for parsers and AI agents
- allow compact binary encoding

9. Trace Records

9.1 Time record

Syntax:

```
T,+<delta>
```

Example:

```
T,+0

T,+1

T,+4
```

Rules:

- delta is relative to the previous trace timestamp
- time starts at 0 unless otherwise stated
- all subsequent records are associated with the current time until the next T

9.2 Single signal delta record

Syntax:

```
D,<signal_id>,<value>
```

Example:

```
D,s0,0x0010

D,s2,EXEC
```

```
D,s5,1
```

9.3 Packed signal delta record

Syntax:

```
P,<signal_id>=<value>[,<signal_id>=<value>]*
```

Example:

```
P,s0=0x0010,s1=0x9123,s2=DECODE
```

This is preferred when multiple signals change at the same timestamp.

9.4 Event record

Syntax:

```
X,<event_type>[,<key>=<value>]*
```

Example:

```
X,inst_decode,name=LOAD,pc=0x0010,rd=r1,addr=0x0123
X,fsm_transition,from=DECODE,to=MEM
X,reset_exit
```

9.5 Assertion record

Syntax:

```
A,<kind>[,<key>=<value>]*
```

Example:

```
A,fail,id=a7,severity=error,msg="rdata contains X",ctx=s3|s4|s7,parent=t100
A,pass,id=a3
```

Recommended keys include id, severity, msg, ctx, parent, checker, and property.

9.6 Snapshot record

Syntax:

```
N,<snap_type>[,<signal_id>=<value>]*
```

Examples:

```
N,full,s0=0x0000,s2=RESET,s5=0
```

```
N,ctx,s2=MEM,s3=0x1000,s4=0xXX0A
```

Purpose:

- checkpointing
- assertion context
- periodic recovery state
- simplified AI debugging windows

9.7 Comment record

Syntax:

C,"free text comment"

10. Transaction Records

Transactions represent meaningful design operations such as instruction execution, bus request or response, AXI burst, memory load or store, cache fill, DMA transfer, interrupt handling, or reset sequencing.

10.1 Transaction lifecycle model

Every transaction may progress through begin, zero or more updates, and end or abort. Transactions may also reference a parent transaction, reference child transactions, bind to relevant signals, and bind to assertions.

10.2 Transaction begin record

Syntax:

```
Q,B,id=<txn_id>,type=<txn_type>[,<key>=<value>]*
```

Example:

```
Q,B,id=i44,type=tt0,pc=0x0010,opcode=0x9123,mnemonic=LOAD,rd=r1
```

```
Q,B,id=t100,type=tt1,addr=0x0123,size=2,parent=i44,agent=cpu0
```

10.3 Transaction update record

Syntax:

```
Q,U,id=<txn_id>[,<key>=<value>]*
```

Examples:

```
Q,U,id=i44,stage=decode
```

```
Q,U,id=t100,phase=address_accepted
```

```
Q,U,id=ax44,beat=3,rdata=0x33,last=0
```

10.4 Transaction end record

Syntax:

```
Q,E,id=<txn_id>,status=<status>[,<key>=<value>]*
```

Examples:

```
Q,E,id=t100,status=ok,rdata=0x00AA,latency=1
```

```
Q,E,id=i44,status=retired,result=0x00AA,latency=3
```

10.5 Transaction abort record

Syntax:

```
Q,X,id=<txn_id>,reason=<reason>[,<key>=<value>]*
```

Example:

```
Q,X,id=t102,reason=timeout
```

10.6 Transaction link record

Syntax:

```
Q,L,parent=<parent_id>,child=<child_id>,relation=<relation>
```

Examples:

```
Q,L,parent=i44,child=t100,relation=caused_by
```

```
Q,L,parent=t20,child=t23,relation=forwarded_to_dram
```

10.7 Transaction summary record

Syntax:

```
QS,<key>=<value>[,<key>=<value>]*
```

Examples:

```
QS,time=200,active=3,completed=155,failed=2
```

```
QS,type=mem_read,count=80,avg_latency=6,max_latency=21
```

```
QS,type=inst,count=120,retired=118,trapped=2
```

11. Standard Transaction Types

Recommended canonical transaction names include:

- inst
- branch
- load
- store
- interrupt
- exception
- mem_read
- mem_write
- axi_read
- axi_write
- apb_read
- apb_write
- dma_read
- dma_write
- cache_lookup
- cache_fill
- cache_evict
- writeback
- snoop
- invalidate
- clean
- reset_seq
- power_up
- power_down

- retention_save
- retention_restore

12. Recommended Tags

Recommended tags include:

- clock
- reset
- pc
- instruction
- fsm
- state
- register
- architectural
- bus
- req_addr
- req_data
- rsp_data
- control
- handshake_valid
- handshake_ready
- assertion_context
- memory
- axi
- apb
- interrupt
- power

13. Value Representation

13.1 Numeric values

Use hex for vectors unless decimal is semantically better.

Examples: 0x0010, 0xABCD, 42.

13.2 Enums

Use symbolic names where possible, for example FETCH and MEM.

13.3 Unknowns

Allowed representations include X, Z, and 0xXX0A.

13.4 Strings

Strings should be quoted if containing spaces or punctuation.

14. Correlation Rules

A major feature of XTrace is explicit correlation across layers.

14.1 Assertion to transaction

Use parent=<txn_id> in assertion records.

Example:

```
A,fail,id=a7,severity=error,msg="rdata X during valid response",parent=t100,ctx=s4|s7|s8
```

14.2 Transaction to signals

Use ctx= in transaction begin or update records.

Example:

```
Q,B,id=t100,type=tt1,addr=0x1000,ctx=s3|s4|s7|s8
```

14.3 Instruction to sub-transaction

Use parent=i44 in bus or memory transactions.

Example:

```
Q,B,id=t100,type=tt1,addr=0x0123,parent=i44
```

15. Core Example

```
@xtrace 1.0

@format text

@timescale 1ns
```

```
@design cpu16
@profile architectural

@section dict
M,m0,/top
M,m1,/top/cpu
S,s0,m1,pc,u16,tags=pc|state,enc=delta
S,s1,m1,ir,u16,tags=instruction,enc=abs
S,s2,m1,state,enum:cpu_state,tags=fsm|state,enc=enum
S,s3,m1,mem_addr,u16,tags=req_addr|bus
S,s4,m1,mem_rdata,u16,tags=rsp_data|bus
S,s5,m1,mem_wr,bit,tags=control|bus
S,s6,m1,r1,u16,tags=register|architectural
E,cpu_state,0=RESET,1=FETCH,2=DECODE,3=EXEC,4=MEM,5=WB
TQ,tt0,inst,fields=pc|opcode|mnemonic|rd|result|status|latency,tags=instruction
TQ,tt1,mem_read,fields=addr|size|rdata|status|latency|parent,tags=bus|memory

@section trace
T,+0
N,full,s0=0x0000,s2=RESET,s5=0,s6=0x0000

T,+1
P,s2=FETCH,s0=0x0000
X,inst_fetch,pc=0x0000

T,+1
P,s1=0x9123,s2=DECODE
X,inst_decode,name=LOAD,pc=0x0000,rd=r1,addr=0x0123

T,+1
P,s2=MEM,s3=0x0123,s5=0
```

```
T,+1
P,s4=0x00AA

T,+1
P,s2=WB,s6=0x00AA

@section transactions
Q,B,id=i44,type=tt0,pc=0x0000,opcode=0x9123,mnemonic=LOAD,rd=r1,ctx=s0|s1|s2|s6
Q,U,id=i44,stage=decode
Q,B,id=t100,type=tt1,addr=0x0123,size=2,parent=i44,agent=cpu0,ctx=s3|s4|s5
Q,E,id=t100,status=ok,rdata=0x00AA,latency=1
Q,U,id=i44,stage=wb
Q,E,id=i44,status=retired,result=0x00AA,latency=3

@section end
```

16. Text Grammar (BNF)

```
file            := header section+
header          := version_decl format_decl? timescale_decl? design_decl? profile_decl?
version_decl    := "@xtrace" version
format_decl     := "@format" ("text" | "binary")
timescale_decl  := "@timescale" token
design_decl     := "@design" token
profile_decl    := "@profile" token
section         := dict_section | trace_section | txn_section | end_section

dict_section    := "@section dict" newline dict_record+
trace_section   := "@section trace" newline trace_record+
txn_section     := "@section transactions" newline txn_record+
```

```
end_section     := "@section end"


dict_record     := module_rec | signal_rec | enum_rec | txn_schema_rec

module_rec      := "M," id "," path

signal_rec      := "S," id "," id "," name "," type attrs*

enum_rec        := "E," name "," enum_item ("," enum_item)*

txn_schema_rec  := "TQ," id "," name "," attrs*


trace_record    := time_rec | delta_rec | packed_rec | event_rec | assert_rec | snap_rec |
comment_rec

time_rec        := "T,+" integer

delta_rec       := "D," id "," value

packed_rec      := "P," assign ("," assign)*

event_rec       := "X," name ("," kvpair)*

assert_rec      := "A," name ("," kvpair)*

snap_rec        := "N," name ("," assign)*

comment_rec     := "C," quoted_string


txn_record      := txn_begin | txn_update | txn_end | txn_abort | txn_link | txn_summary

txn_begin       := "Q,B," kvpair ("," kvpair)*

txn_update      := "Q,U," kvpair ("," kvpair)*

txn_end         := "Q,E," kvpair ("," kvpair)*

txn_abort       := "Q,X," kvpair ("," kvpair)*

txn_link        := "Q,L," kvpair ("," kvpair)*

txn_summary     := "QS," kvpair ("," kvpair)*
```

17. Semantic Model

17.1 General semantics

XTrace is a simulator or tool evidence stream, not merely a rendered waveform dump. Records are interpreted in sequence. The current simulation context consists of the active time, and optionally additional profile-defined context such as cycle domain or scheduler region.

17.2 Dictionary semantics

Dictionary records define stable identities and metadata. Once introduced, an ID retains its meaning throughout the file.

17.3 Time semantics

A T record advances or rebinds the current time context. All subsequent D, P, X, A, N, and Q records inherit the most recent time until another T record appears.

17.4 Signal semantics

A D or P record changes the current value of one or more signals in the reconstructed state model. Signal values persist until explicitly changed.

17.5 Event semantics

An X record annotates semantically meaningful activity but does not itself require a persistent state update unless a consumer chooses to derive one.

17.6 Assertion semantics

An A record represents assertion, checker, or property outcomes. An A,fail record is intended to be directly consumable by a parser or AI agent and may carry context signals, parent linkage, source mapping, and severity.

17.7 Snapshot semantics

An N record materializes a compact state image. N,full is intended as a checkpoint. N,ctx is intended as a local evidence packet around a failure, event, or transaction boundary.

17.8 Transaction semantics

A transaction ID is created by Q,B. It remains live until Q,E or Q,X. Q,U refines or extends the transaction state. Parent-child links form a causality graph across instructions, bus activity, memory requests, cache activity, and failures.

17.9 Correlation semantics

The format supports explicit links among transactions, assertions, signals, and higher-level events. Consumers should prefer explicit links over heuristic inference when both are available.

18. Binary Model

Recommended packet classes:

- 0x01 module dictionary
- 0x02 signal dictionary
- 0x03 enum dictionary
- 0x04 transaction schema dictionary
- 0x10 time delta
- 0x11 signal delta
- 0x12 packed signal delta block
- 0x20 event
- 0x21 assertion
- 0x22 snapshot
- 0x40 transaction begin
- 0x41 transaction update
- 0x42 transaction end
- 0x43 transaction abort
- 0x44 transaction link
- 0x45 transaction summary

Recommended primitive encodings:

- ULEB128 for IDs and time deltas
- zigzag varint for signed values
- string table for repeated strings
- optional zstd block compression

19. Compression Recommendations

To keep the format compact:

1. use dictionary IDs for repeated names

2. use time deltas rather than absolute times

3. use packed signal records when multiple signals change

4. prefer transaction records over dumping every intermediate net

5. emit periodic snapshots only when useful

6. dump architectural or semantic signals by default

7. keep microarchitectural signals profile-gated

8. use enum coding for FSM states

9. allow XOR or arithmetic delta encoding for counters and PC

10. compress the binary stream with zstd

20. Emitter Model

Recommended emission sources:

- simulator core
- DPI-C callback
- UVM monitor
- generated instrumentation module
- custom debug interface in a homegrown simulator

Recommended sampling points:

- positive clock edge
- bus handshake completion
- instruction retire

- assertion trigger
- reset or power transitions

Emitter flow:

1. initialize dictionary

2. assign stable signal IDs

3. track previous values

4. emit T,+delta

5. emit P,... for changed watched signals

6. emit Q,... for transactions

7. emit A,... for assertions

8. emit N,ctx,... on important failures

SystemVerilog-style pseudo-emitter:

```
always_ff @(posedge clk) begin
  xtrace_time_advance(1);

  if (pc != pc_prev || ir != ir_prev || state != state_prev) begin
    xtrace_pack_begin();
    if (pc    != pc_prev)    xtrace_pack_signal("s0", pc);
    if (ir    != ir_prev)    xtrace_pack_signal("s1", ir);
    if (state != state_prev) xtrace_pack_signal("s2", state.name());
```

```
    xtrace_pack_end();
  end

  if (inst_decode_valid) begin
    xtrace_event("inst_decode",
      $sformatf("name=%s,pc=0x%0h,rd=%s", mnemonic, pc, rd_name));
  end

  if (mem_req_fire) begin
    xtrace_txn_begin("t100", "mem_read",
      $sformatf("addr=0x%0h,size=%0d,parent=%s", mem_addr, 2, inst_id));
  end

  if (mem_rsp_fire) begin
    xtrace_txn_end("t100",
      $sformatf("status=ok,rdata=0x%0h,latency=%0d", mem_rdata, mem_lat));
  end
end
```

21. Parser-Facing Normalized Model

A parser should expose at least these internal objects.

21.1 Signal dictionary example

```
{
  "s0": {"path":"/top/cpu/pc","type":"u16","tags":["pc","state"]},
  "s1": {"path":"/top/cpu/ir","type":"u16","tags":["instruction"]}
}
```

21.2 Current signal state example


```
{
  "time": 101,
  "signals": {
    "s0": "0x0010",
    "s2": "MEM"
    }
}
```


21.3 Transaction object example


```
{
  "id": "t100",
  "type": "mem_read",
  "start_time": 101,
  "end_time": 102,
  "latency": 1,
  "parent": "i44",
  "status": "ok",
  "addr": "0x0123",
  "rdata": "0x00AA"
}
```


This normalized representation is the preferred ingestion form for an AI agent.

22. Compliance Levels

Level 0: minimal

- dictionary
- time
- signal deltas

Level 1: semantic

- level 0 plus events and assertions

Level 2: transactional

- level 1 plus transaction lifecycle and parent-child linkage

Level 3: AI-native

- level 2 plus signal tags, context snapshots, summaries, and transaction schemas

Recommended target: Level 3.

Appendix A. Xezim Native Profile

A.1 Purpose

The Xezim Native Profile defines how Xezim should generate XTrace records directly from simulation. It keeps the core specification generic while allowing Xezim to provide a richer AI-debug stream.

The profile name is:

```
@profile xezim_ai_debug
```

The producer declaration is:

```
@producer xezim <version>
```

Example header:

```
@xtrace 1.0
@format text
@producer xezim 0.1.0
@timescale 1ps
@design openc910_subsystem
@profile xezim_ai_debug
```

```
@capabilities signal_delta|transactions|assertions|source_map|sv_region|context_bundle|
index_blocks
```

Implementation modes:

- raw_delta: dictionary plus time plus signal deltas only
- semantic: raw_delta plus events, assertions, FSM transitions, and source locations
- ai_debug: semantic plus transactions, causality links, context bundles, summaries, and AI chunking metadata

Recommended Xezim default: ai_debug.

A.2 Header extensions

New optional top-level directives:

```
@producer <tool_name> <tool_version>
```

```
@capabilities <capability_list>
```

Recommended capability names:

- signal_delta
- transactions
- assertions
- events
- source_map
- sv_region
- delta_cycle
- context_bundle
- index_blocks
- binary_blocks
- ai_chunks
- compression_zstd

A.3 Source map extension

New records:

```
F,<file_id>,<path>[,sha=<sha256>]
```

```
SL,<object_id>,file=<file_id>,line=<line>[,col=<col>][,decl=<name>]
```

```
EL,<object_id>,kind=<kind>,src=<source_object>,elab=<elab_path>
```

Examples:

```
F,f0,rtl/cpu.sv,sha=8d41...

SL,s12,file=f0,line=118,decl=pc_q

SL,a7,file=f0,line=241,decl=p_mem_rsp_known

EL,m5,kind=generate_instance,src=gen_core[0],elab=/top/u_cluster/gen_core[0]/u_cpu
```

A.4 SystemVerilog time, delta, and region semantics

Updated time record:

```
T,+<time_delta>[,d=<delta_index>][,r=<region>]
```

Examples:

```
T,+10,d=0,r=active

T,+0,d=1,r=nba

T,+0,d=2,r=observed
```

Recommended region names:

- preponed
- active
- inactive
- nba
- observed
- reactive
- postponed

Semantic rule:

A record inherits current time, delta index, and region from the most recent T record. If d and r are omitted, d=0 and r=active are assumed.

A.5 Clock and cycle domain extension

Dictionary record:

```
CD,<clock_id>,signal=<signal_id>,edge=<posedge|negedge|both>[,domain=<name>]
```

Cycle marker:

```
CY,<clock_id>,+<cycle_delta>[,edge=<edge>]
```

Example:

```
CD,c0,signal=s_clk,edge=posedge,domain=core

CY,c0,+1,edge=posedge
```

CY does not replace T. It provides a cycle-domain overlay.

A.6 4-state value semantics

XTrace must preserve SystemVerilog 4-state logic.

Allowed text forms:

- 1
- 0
- X
- Z
- 0x10AF
- 0b10XZ_0011
- 4s:bits=1001,xmask=0010,zmask=0100

Recommended Xezim text output:

- use hex for fully known values
- use 0b with X or Z when unknowns are sparse and readability matters
- use 4s:bits/xmask/zmask when compact exact representation is needed

A.7 Context bundle extension

Context bundle records:

```
CB,B,id=<bundle_id>,reason=<reason>,parent=<record_id>,window=<before>:<after>,sigset=<sig
nals>

CB,E,id=<bundle_id>[,summary=<text>]
```

Example:

```
A,fail,id=a7,severity=error,msg="rdata contains X during valid
response",parent=t100,ctx=s2|s3|s4|s7|s8

CB,B,id=cb7,reason=assert_fail,parent=a7,window=-5:+2,sigset=s2|s3|s4|s7|s8
```

```
T,+0,d=0,r=observed

N,ctx,s2=MEM,s3=0x1000,s4=0b10XX_1010,s7=1,s8=1

Q,U,id=t100,phase=response_observed

CB,E,id=cb7,summary="Memory response was valid but rdata contained unknown bits."
```

A context bundle is a compact evidence packet and need not contain every signal transition.

A.8 Xezim transaction dump policy

Transactions should come from one of three sources:

1. built-in Xezim monitors for common protocols

2. user-defined monitor plugins

3. explicit SystemVerilog system-task calls

Recommended transaction types for Xezim include inst, pipeline_stage, mem_read, mem_write, bus_req, bus_rsp, axi_read, axi_write, apb_read, apb_write, cache_lookup, cache_fill, cache_evict, interrupt, exception, and reset_seq.

Recommended Xezim-specific transaction fields:

- src=<file_id>:<line>
- region=<sv_region>
- delta=<delta_index>
- clock=<clock_id>
- cycle=<cycle_number>
- ctx=<signal_set>
- elab=<elaborated_scope>
- parent=<causal_parent>

A.9 Xezim SystemVerilog system-task API

Recommended tasks:

$xtrace_watch(signal, "tag1|tag2");

$xtrace_event("event_type", "k1=v1,k2=v2");

$xtrace_assert("id", pass, "severity=error,msg=...");

$xtrace_txn_begin("id", "type", "k1=v1,k2=v2");

$xtrace_txn_update("id", "k1=v1,k2=v2");

$xtrace_txn_end("id", "status=ok,k1=v1");

$xtrace_txn_abort("id", "reason=timeout");

$xtrace_link("parent", "child", "relation=caused_by");

$xtrace_context("id", "reason=assert_fail,parent=a7,signals=s1|s2");

These system tasks should write into the same XTrace stream as native Xezim scheduler records, preserving current time, delta, region, and source context.

A.10 Xezim CLI proposal

Examples:

```
xezim sim top.sv --top top --xtrace out.xt --xtrace-profile ai_debug --xtrace-source-map
--xtrace-regions --xtrace-clock top.clk --xtrace-context-window 10 --xtrace-transactions
monitors.toml
```

```
xezim sim top.sv --xtrace out.xt --xtrace-profile raw_delta
```

```
xezim sim top.sv --xtrace out.xtb --xtrace-format binary --xtrace-compress zstd
```

```
xezim vcd2xtrace input.vcd -o output.xt --profile raw_delta
```

```
xezim vcd2xtrace input.vcd -o output.xt --profile semantic --rules valid_ready_rules.toml
```

Native Xezim emission should be the preferred path. vcd2xtrace is a compatibility bridge.

A.11 Monitor rule file proposal

Example monitors.toml:

```
[[monitor]]

name = "core_mem"
```

```
type = "valid_ready"

scope = "/top/cpu"

valid = "mem_valid"

ready = "mem_ready"

addr  = "mem_addr"

wdata = "mem_wdata"

rdata = "mem_rdata"

write = "mem_write"

txn_read_type  = "mem_read"

txn_write_type = "mem_write"


[[monitor]]

name = "retire"

type = "instruction_retire"

scope = "/top/cpu"

valid = "retire_valid"

pc = "retire_pc"

opcode = "retire_opcode"

rd = "retire_rd"

result = "retire_result"
```

A.12 AI chunking and index blocks

New records:

```
IB,B,id=<block_id>,time=<start_time>,cycle=<start_cycle>

IB,E,id=<block_id>,records=<count>,sha=<sha256>[,summary=<text>]
```

Example:

```
IB,B,id=b120,time=100000,cycle=5000

...

IB,E,id=b120,records=8421,sha=92af...,summary="Core executed 110 instructions, 2 memory
retries, no assertion failures."
```

Index blocks are chunk boundaries that help retrieval-oriented tools and AI agents access relevant intervals without reading the entire dump.

A.13 Xezim-native example

```
@xtrace 1.0

@format text

@producer xezim 0.1.0

@timescale 1ps

@design cpu16

@profile xezim_ai_debug

@capabilities signal_delta|transactions|assertions|source_map|sv_region|context_bundle


@section dict

F,f0,rtl/cpu16.sv,sha=8d41...

M,m0,/top

M,m1,/top/cpu

S,s0,m1,clk,bit,tags=clock

S,s1,m1,rst_n,bit,tags=reset

S,s2,m1,pc,u16,tags=pc|state,enc=delta

S,s3,m1,ir,u16,tags=instruction,enc=abs

S,s4,m1,state,enum:cpu_state,tags=fsm|state,enc=enum

S,s5,m1,mem_addr,u16,tags=req_addr|bus

S,s6,m1,mem_rdata,u16,tags=rsp_data|bus

CD,c0,signal=s0,edge=posedge,domain=core

E,cpu_state,0=RESET,1=FETCH,2=DECODE,3=EXEC,4=MEM,5=WB

TQ,tt0,inst,fields=pc|opcode|mnemonic|rd|result|status|latency_cycles,tags=instruction

TQ,tt1,mem_read,fields=addr|size|rdata|status|latency_cycles|parent,tags=bus|memory

SL,s2,file=f0,line=44,decl=pc_q

SL,s4,file=f0,line=52,decl=state_q


@section trace
```

```
T,+0,d=0,r=active
N,full,s1=0,s2=0x0000,s4=RESET
T,+1000,d=0,r=active
P,s1=1
X,reset_exit,clock=c0
CY,c0,+1,edge=posedge
P,s4=FETCH,s2=0x0000
X,inst_fetch,pc=0x0000

T,+1000,d=0,r=active
CY,c0,+1,edge=posedge
P,s3=0x9123,s4=DECODE
Q,B,id=i44,type=tt0,pc=0x0000,opcode=0x9123,mnemonic=LOAD,rd=r1,ctx=s2|s3|s4

T,+1000,d=0,r=active
CY,c0,+1,edge=posedge
P,s4=MEM,s5=0x0123
Q,B,id=t100,type=tt1,addr=0x0123,size=2,parent=i44,agent=cpu0,ctx=s5|s6

T,+0,d=1,r=nba
P,s6=0x00AA

T,+0,d=2,r=observed
Q,E,id=t100,status=ok,rdata=0x00AA,latency_cycles=1
Q,E,id=i44,status=retired,result=0x00AA,latency_cycles=3

@section end
```

Appendix B. Final Definition

XTrace v1.0 is a dictionary-based, delta-encoded, semantic and transactional hardware debug trace format designed for efficient parser implementation and direct AI-agent reasoning.